LAB_3 - Linear Data Structures

A2 - S4 - 2025/2026 - II.2415

Advanced Algorithms and Programming

Team 04

Damiano Spadaccini - Nicolò Romanelli

GitHub repository: https://github.com/damspada/AAP-T04-Semester-Project (branch LAB_03_LinearDS, folder LAB_03)

This is the final version of our LAB_3 report. We kept the pseudo-code and the answers of the report written during the session (T04_LAB3_AES). During the session we did not finish everything, so we added the missing functions and the missing answers. For each exercise we also added more complexity analysis, a short reflection and the tests of our Python code, and at the end the final question.

The code is on GitHub (link above). Each file can be run alone and prints the result of every test.

EXERCISE 1 - CONTENT FEED NAVIGATION WITH DOUBLY LINKED LIST

NULL is the empty pointer. A new node always has next = NULL and prev = NULL.

```
Structure StoryNode
    story_id: Integer
    user_id: Integer
    content_preview: String
    timestamp: Integer
    views: Integer
    next: pointer to StoryNode
    prev: pointer to StoryNode
End Structure

Structure DoublyLinkedList
    head: pointer to StoryNode
    tail: pointer to StoryNode
    current: pointer to StoryNode
    size: Integer
End Structure

Procedure AddStory(output L: DoublyLinkedList, node: pointer to StoryNode)
    node.prev <- L.tail
    node.next <- NULL
    If L.tail = NULL Then
        // empty feed: the new story is also the first one and the current one
        L.head <- node
        L.current <- node
    Else
        L.tail.next <- node
    End If
    L.tail <- node
    L.size <- L.size + 1
End

Function Find(L: DoublyLinkedList, story_id: Integer) -> pointer to StoryNode
    p <- L.head
    While p != NULL And p.story_id != story_id Do
        p <- p.next
    End While
    // NULL if the story is not in the feed
    Return p
End
```

```
// p must be a node of L
Procedure Unlink(output L: DoublyLinkedList, p: pointer to StoryNode)
    If p.prev = NULL Then
        L.head <- p.next
    Else
        p.prev.next <- p.next
    End If
    If p.next = NULL Then
        L.tail <- p.prev
    Else
        p.next.prev <- p.prev
    End If
    p.next <- NULL
    p.prev <- NULL
    L.size <- L.size - 1
End

Function RemoveStory(output L: DoublyLinkedList, story_id: Integer) -> Boolean
    removed <- False
    p <- Find(L, story_id)
    If p != NULL Then
        // if we remove the current story, current goes to the next one (or to the previous one at the end)
        If L.current = p Then
            If p.next != NULL Then
                L.current <- p.next
            Else
                L.current <- p.prev
            End If
        End If
        Unlink(L, p)
        removed <- True
    End If
    Return removed
End

Function MoveForward(output L: DoublyLinkedList) -> pointer to StoryNode
    // no next story
    If L.current = NULL Or L.current.next = NULL Then
        Return NULL
    End If
    L.current <- L.current.next
    Return L.current
End

// MoveBackward is the same as MoveForward, with prev instead of next

Function JumpTo(output L: DoublyLinkedList, story_id: Integer) -> pointer to StoryNode
    p <- Find(L, story_id)
    // current does not change if the story is not found
    If p != NULL Then
        L.current <- p
    End If
    Return p
End
```

```
Function InsertAfter(output L: DoublyLinkedList, current_id: Integer, new_story: pointer to StoryNode)
    -> Boolean
    inserted <- False
    p <- Find(L, current_id)
    If p != NULL Then
        new_story.prev <- p
        new_story.next <- p.next
        If p.next = NULL Then
            // p was the last story
            L.tail <- new_story
        Else
            p.next.prev <- new_story
        End If
        p.next <- new_story
        L.size <- L.size + 1
        inserted <- True
    End If
    Return inserted
End

Procedure TrackView(output L: DoublyLinkedList)
    If L.current != NULL Then
        L.current.views <- L.current.views + 1
    End If
End

Function MostViewed(L: DoublyLinkedList) -> pointer to StoryNode
    best <- L.head
    p <- L.head
    While p != NULL Do
        If p.views > best.views Then
            best <- p
        End If
        p <- p.next
    End While
    Return best
End

// each time we take the most viewed story of the old list and we add it at the end of a new list
Procedure ReorderByViews(output L: DoublyLinkedList)
    // we keep the same current story
    cur <- L.current
    N <- empty DoublyLinkedList
    While L.head != NULL Do
        best <- MostViewed(L)
        Unlink(L, best)
        AddStory(N, best)
    End While
    L.head <- N.head
    L.tail <- N.tail
    L.size <- N.size
    L.current <- cur
End
```

Added after the session: DisplayAroundCurrent.

```
Procedure DisplayAroundCurrent(L: DoublyLinkedList, k: Integer)
    If L.current != NULL Then
        // go back k stories, or stop at the first one
        start <- L.current
        before <- 0
        While before < k And start.prev != NULL Do
            start <- start.prev
            before <- before + 1
        End While
        // show the stories before, the current one and k stories after (or stop at the last one)
        p <- start
        count <- 0
        While p != NULL And count < before + 1 + k Do
            Print(p.story_id, p.content_preview)
            p <- p.next
            count <- count + 1
        End While
    End If
End
```

1. Compare time complexity of forward/backward navigation in singly vs. doubly linked lists

Moving forward is O(1) in both lists, because every node has the pointer next. Moving backward is O(1) in a doubly linked list, thanks to prev, but O(n) in a singly linked list, because we have to start again from the head to find the previous node.

2. What is the time complexity of removing a node given only a pointer to that node?

O(1) in a doubly linked list: with prev and next we connect the previous node and the next node directly, like in Unlink. In a singly linked list it is O(n), because we first have to find the previous node starting from the head.

3. How much extra memory per node does a doubly linked list require compared to singly linked?

One pointer more per node (prev), so 8 bytes on a 64-bit machine. For n nodes this is O(n) extra memory.

4. In what scenarios would the bidirectional navigation justify the memory overhead?

When users often go back: a story viewer, a music playlist, the history of a browser or an undo/redo list. It also helps when we often remove stories in the middle of the feed. In our case one pointer is small compared to the data of a story, so the overhead is worth it.

More complexity analysis and reflection

AddStory, MoveForward, MoveBackward and TrackView are O(1), because they only change a few pointers. RemoveStory, JumpTo and InsertAfter are O(n), because Find has to search the story from the head. DisplayAroundCurrent is O(k), MostViewed is O(n) and ReorderByViews is O(n^2), because it calls MostViewed for every story. The memory is O(n) for n stories.

If we remove the current story, current goes to the next one, so the user can continue to scroll. ReorderByViews is simple but slow: for a long feed a merge sort on the list would be better, O(n log n).

Test set

The tests use the example of the lab and some stories with Italian places. After every insert, remove and reorder, the code also reads the list backwards with prev, to check that the links are right. All the tests print OK.

Case	Operations	Expected	Got
lab example	Story2 viewed 2 times, Story3 1 time, then MostViewed	Story2	Story2
lab example	ReorderByViews	[2, 3, 1]	[2, 3, 1]
lab example	current after ReorderByViews	Story3	Story3
first story	MoveBackward	NULL	NULL
last story	MoveForward	NULL, current does not change	NULL, current does not change

Case	Operations	Expected	Got
story not in the feed	JumpTo(9)	NULL, current does not change	NULL, current does not change
story not in the feed	InsertAfter(9, new story)	False	False
after the last story	InsertAfter(3, Story4)	tail = Story4	tail = Story4
remove the current story	current = Story2, RemoveStory(2)	current = Story3	current = Story3
remove the current story at the end	current = Story4 (last), RemoveStory(4)	current = Story3	current = Story3
story not in the feed	RemoveStory(9)	False	False
display in the middle	7 stories, current = 4, DisplayAroundCurrent(2)	2, 3, 4, 5, 6	2, 3, 4, 5, 6
display near the start	current = 2, DisplayAroundCurrent(3)	1, 2, 3, 4, 5	1, 2, 3, 4, 5
k = 0	current = 2, DisplayAroundCurrent(0)	2	2
k bigger than the feed	DisplayAroundCurrent(10)	all 7 stories	all 7 stories
empty feed	MoveForward, MostViewed	NULL, NULL	NULL, NULL
remove the only story	RemoveStory(1)	head, tail, current = NULL, size = 0	head, tail, current = NULL, size = 0
equal views	views 3, 1, 3, 0, 1, then ReorderByViews	[1, 3, 2, 5, 4]	[1, 3, 2, 5, 4]

EXERCISE 2 - ACTIVITY FEED PROCESSING WITH STACKS AND QUEUES

Both structures use linked nodes: each node has a value (an activity or a notification, as a String) and a pointer next.

PART A - RECENT ACTIVITY STACK

```
Structure Node
    value: String
    next: pointer to Node
End Structure

Structure ActivityStack
    top: pointer to Node
    count: Integer
End Structure

Procedure Push(output S: ActivityStack, activity: String)
    node <- new Node
    node.value <- activity
    // the new node goes on top of the old top
    node.next <- S.top
    S.top <- node
    S.count <- S.count + 1
End

Function Pop(output S: ActivityStack) -> String
    If S.top = NULL Then
        Return NULL
    End If
    activity <- S.top.value
    S.top <- S.top.next
    S.count <- S.count - 1
    Return activity
End

// Peek returns S.top.value without removing it, IsEmpty checks S.top = NULL and Size returns S.count

// history = the actions done, undo_stack = the actions undone that can be redone
Structure ActivityHistory
    history: ActivityStack
    undo_stack: ActivityStack
End Structure

Function UndoLast(output H: ActivityHistory) -> String
    activity <- Pop(H.history)
    If activity != NULL Then
        // Revert does the opposite action: like -> unlike, follow -> unfollow
        Revert(activity)
        Push(H.undo_stack, activity)
    End If
    Return activity
End
```

Added after the session: DisplayRecent, DoActivity and Redo.

```
Procedure DisplayRecent(S: ActivityStack, n: Integer)
    p <- S.top
    i <- 0
    // from the most recent, stop after n activities or at the bottom of the stack
    While p != NULL And i < n Do
        Print(p.value)
        p <- p.next
        i <- i + 1
    End While
End

Procedure DoActivity(output H: ActivityHistory, activity: String)
    Push(H.history, activity)
    // after a new action the old undone actions can not be redone anymore
    H.undo_stack <- empty ActivityStack
End

Function Redo(output H: ActivityHistory) -> String
    activity <- Pop(H.undo_stack)
    If activity != NULL Then
        // Apply does the action again
        Apply(activity)
        Push(H.history, activity)
    End If
    Return activity
End
```

PART B - NOTIFICATION QUEUE

```
Structure NotificationQueue
    head: pointer to Node
    rear: pointer to Node
    count: Integer
End Structure

Procedure Enqueue(output Q: NotificationQueue, notification: String)
    node <- new Node
    node.value <- notification
    node.next <- NULL
    If Q.rear = NULL Then
        // empty queue
        Q.head <- node
    Else
        Q.rear.next <- node
    End If
    Q.rear <- node
    Q.count <- Q.count + 1
End
```

```
Function Dequeue(output Q: NotificationQueue) -> String
    notification <- NULL
    If Q.head != NULL Then
        notification <- Q.head.value
        Q.head <- Q.head.next
        If Q.head = NULL Then
            // the queue is empty now
            Q.rear <- NULL
        End If
        Q.count <- Q.count - 1
    End If
    Return notification
End

// the urgent notification goes at the front, so it is the next one to be processed
Procedure PriorityEnqueue(output Q: NotificationQueue, notification: String)
    node <- new Node
    node.value <- notification
    node.next <- Q.head
    Q.head <- node
    If Q.rear = NULL Then
        // the queue was empty, so the new node is also the last one
        Q.rear <- node
    End If
    Q.count <- Q.count + 1
End

Added after the session: DisplayPending.

// Front, IsEmpty and Size are like Peek, IsEmpty and Size of the stack, with Q.head instead of S.top
Procedure DisplayPending(Q: NotificationQueue)
    p <- Q.head
    While p != NULL Do
        Print(p.value)
        p <- p.next
    End While
End
```

PART C - INTEGRATED FEED PROCESSING

```
Structure FeedProcessor
    recent_activities: ActivityStack
    notification_queue: NotificationQueue
    processed_log: NotificationQueue
End Structure

Procedure ProcessIncoming(output F: FeedProcessor)
    If F.notification_queue.head != NULL Then
        // the oldest notification goes on top of the recent stack
        Push(F.recent_activities, Dequeue(F.notification_queue))
    End If
End

Procedure BatchProcess(output F: FeedProcessor, k: Integer)
    i <- 0
    // stop after k notifications or when the queue is empty
    While i < k And F.notification_queue.head != NULL Do
        ProcessIncoming(F)
        i <- i + 1
    End While
End
```

```
Procedure ClearHistory(output F: FeedProcessor)
    // the most recent activity goes first in the log
    While F.recent_activities.top != NULL Do
        Enqueue(F.processed_log, Pop(F.recent_activities))
    End While
End

// GetStats returns the three counters: recent_activities.count, notification_queue.count,
    processed_log.count
```

1. Compare time complexity of all operations

Push, Pop, Peek, IsEmpty, Size and UndoLast are O(1), because we only work on the top and we keep a counter. Enqueue, Dequeue and PriorityEnqueue are O(1) too: Enqueue thanks to the pointer rear, the others because they work on the head. ProcessIncoming and GetStats are O(1), BatchProcess(k) is O(k) and ClearHistory is O(m), with m the size of the recent stack.

2. For a queue, which implementation (array circular buffer vs. linked list) is more memory efficient for unknown size?

The linked list. It uses memory only for the notifications really in the queue, plus one pointer per element. A circular buffer needs a fixed capacity: if it is too big the memory is wasted, if it is too small we have to copy everything into a bigger array.

3. What is the amortized time complexity of a sequence of n enqueue/dequeue operations?

O(1) per operation, so O(n) for the whole sequence. With a linked list every enqueue and dequeue is O(1) also in the worst case. With a circular buffer that doubles its size when it is full, one copy costs O(n), but it happens rarely, so the total stays O(n).

4. How would you implement a deque (double-ended queue) to support both stack and queue operations?

With a doubly linked list with head and tail: we can add and remove at both ends in O(1). Used as a stack we add and remove at the same end, used as a queue we add at one end and remove at the other one. Another way is a circular buffer with two indices, one for the front and one for the back.

More complexity analysis and reflection

All the operations on the top of the stack and on the head or the rear of the queue are O(1). DisplayRecent(n), DisplayPending and ClearHistory are O(n), because they read or move every element. The memory is O(n) for n activities or notifications, with one pointer for each node.

PriorityEnqueue is simple, but if more urgent notifications arrive, the last one is processed first. To keep the arrival order also between the urgent notifications, we could use a second queue only for them.

Test set

All the tests print OK. In the code the activities are strings like "like photo of Marco", and Revert changes the first word (like -> unlike).

Case	Operations	Expected	Got
empty stack	Pop, Peek	NULL, NULL	NULL, NULL
stack	3 Push, then Peek and Size	last activity, 3	last activity, 3
display	DisplayRecent(2)	the 2 most recent	the 2 most recent
n bigger than the stack	DisplayRecent(10) with 3 activities	3 activities	3 activities
nothing to undo	UndoLast	NULL	NULL
undo	2 DoActivity, then UndoLast	the last activity, history has 1	the last activity, history has 1
redo	Redo	the activity is back on top	the activity is back on top
redo after a new action	UndoLast, DoActivity, Redo	NULL	NULL
empty queue	Dequeue, Front	NULL, NULL	NULL, NULL
queue	3 Enqueue, then Dequeue	"New follower" (the oldest)	"New follower"
queue emptied	3 Dequeue, then Enqueue	rear = NULL, then Enqueue works	rear = NULL, then Enqueue works

Case	Operations	Expected	Got
urgent on an empty queue	PriorityEnqueue	head = rear = the urgent one	head = rear = the urgent one
two urgent	PriorityEnqueue A, Enqueue B, PriorityEnqueue C	C, A, B	C, A, B
lab example	ProcessIncoming	"New follower" on top of the stack, queue: New like, New comment	same
fewer than k	BatchProcess(5) with 2 notifications	GetStats = (6, 0, 0)	(6, 0, 0)
clear history	ClearHistory	GetStats = (0, 0, 6), most recent first in the log	(0, 0, 6), most recent first
empty queue	ProcessIncoming	nothing changes	nothing changes

EXERCISE 3 - ENGAGEMENT-BASED PRIORITY QUEUE

The queue is a singly linked list sorted by engagement_score, from the highest to the lowest, so the head is always the post with the highest priority.

```
Structure Post
    post_id: Integer
    user_id: Integer
    content: String
    timestamp: Integer
    likes: Integer
    comments: Integer
    shares: Integer
    engagement_score: Real
    next: pointer to Post
End Structure

Structure PriorityQueue
    head: pointer to Post
    count: Integer
End Structure

Function Score(p: pointer to Post) -> Real
    Return p.likes * 1 + p.comments * 2 + p.shares * 3
End

Procedure Enqueue(output Q: PriorityQueue, p: pointer to Post)
    // p goes before the first post with a smaller score
    If Q.head = NULL Or p.engagement_score > Q.head.engagement_score Then
        p.next <- Q.head
        Q.head <- p
    Else
        q <- Q.head
        While q.next != NULL And q.next.engagement_score >= p.engagement_score Do
            q <- q.next
        End While
        p.next <- q.next
        q.next <- p
    End If
    Q.count <- Q.count + 1
End

Function DequeueMax(output Q: PriorityQueue) -> pointer to Post
    // NULL if the queue is empty
    p <- Q.head
    If p != NULL Then
        Q.head <- p.next
        p.next <- NULL
        Q.count <- Q.count - 1
    End If
    Return p
End

// PeekMax returns Q.head, IsEmpty checks Q.head = NULL and Size returns Q.count
```

Added after the session: the priority updates (RemoveById, UpdateScore, RefreshAll), the trending window (GetTopK, DecayOlderThan) and the bonus.

```
// takes the post out of the queue and returns it (NULL if it is not in the queue)
Function RemoveById(output Q: PriorityQueue, post_id: Integer) -> pointer to Post
    prev <- NULL
    p <- Q.head
    While p != NULL And p.post_id != post_id Do
        prev <- p
        p <- p.next
    End While
    If p != NULL Then
        If prev = NULL Then
            Q.head <- p.next
        Else
            prev.next <- p.next
        End If
        p.next <- NULL
        Q.count <- Q.count - 1
    End If
    Return p
End
```

```
// new_likes, new_comments and new_shares are added to the old values
Function UpdateScore(output Q: PriorityQueue, post_id: Integer, new_likes: Integer, new_comments:
    Integer, new_shares: Integer) -> Boolean
    updated <- False
    p <- RemoveById(Q, post_id)
    If p != NULL Then
        p.likes <- p.likes + new_likes
        p.comments <- p.comments + new_comments
        p.shares <- p.shares + new_shares
        p.engagement_score <- Score(p)
        // the post goes back in the right position
        Enqueue(Q, p)
        updated <- True
    End If
    Return updated
End
```

```
Procedure RefreshAll(output Q: PriorityQueue)
    p <- Q.head
    // the queue is emptied and every post is inserted again with its new score
    Q.head <- NULL
    Q.count <- 0
    While p != NULL Do
        next_post <- p.next
        p.engagement_score <- Score(p)
        Enqueue(Q, p)
        p <- next_post
    End While
End
```

```
Function GetTopK(Q: PriorityQueue, k: Integer) -> Array of Post
    result <- empty array
    p <- Q.head
    i <- 0
    // there can be fewer than k posts
    While p != NULL And i < k Do
        Append(result, p)
        p <- p.next
        i <- i + 1
    End While
    Return result
End

// factor is between 0 and 1, for example 0.8 to remove 20% of the score
Procedure DecayOlderThan(output Q: PriorityQueue, t: Integer, factor: Real)
    p <- Q.head
    While p != NULL Do
        If p.timestamp < t Then
            p.engagement_score <- p.engagement_score * factor
        End If
        p <- p.next
    End While
    // the order can be wrong now, so every post is inserted again
    p <- Q.head
    Q.head <- NULL
    Q.count <- 0
    While p != NULL Do
        next_post <- p.next
        Enqueue(Q, p)
        p <- next_post
    End While
End
```

BONUS - DUAL-PRIORITY QUEUE

```
Structure DualQueue
    viral_queue: PriorityQueue
    normal_queue: PriorityQueue
End Structure

Procedure DualEnqueue(output D: DualQueue, p: pointer to Post)
    If p.engagement_score > 100 Then
        Enqueue(D.viral_queue, p)
    Else
        Enqueue(D.normal_queue, p)
    End If
End

// Random() gives a real number between 0 and 1
Function DualDequeue(output D: DualQueue) -> pointer to Post
    // if one queue is empty we take from the other one
    If IsEmpty(D.viral_queue) Then
        p <- DequeueMax(D.normal_queue)
    Else If IsEmpty(D.normal_queue) Then
        p <- DequeueMax(D.viral_queue)
    Else If Random() < 0.7 Then
        p <- DequeueMax(D.viral_queue)
    Else
        p <- DequeueMax(D.normal_queue)
    End If
    Return p
End
```

1. Compare time complexity of enqueue for: sorted linked list implementation, unsorted linked list with find-max each time

Sorted list: Enqueue is O(n), because we search the right position, but DequeueMax and PeekMax are O(1), because the maximum is always the head. Unsorted list: Enqueue is O(1), because we add the post at the head, but every DequeueMax or PeekMax is O(n), because we have to read the whole list to find the maximum.

2. What is the worst-case time complexity of update_score? How could you optimize it?

O(n): finding and removing the post is O(n) and inserting it again is O(n). To make it faster we can keep a dictionary post_id -> node, so we find the post in O(1) on average, and use a doubly linked list to remove it in O(1). The new insertion is still O(n) in the worst case; with a heap or a balanced tree everything becomes O(log n).

3. For a social network with millions of posts, would a sorted linked list be practical? Why or why not?

No. Every Enqueue can walk the whole list, so with 10^6 posts this is up to 10^6 steps for a single new post, and the scores change all the time.

4. Space complexity: How much extra memory per node does a priority queue require compared to a standard linked list?

No extra pointer: each node has only next, like a standard singly linked list. The only extra thing is the field engagement_score, one number per post, which we store so we do not compute it again at every comparison. In total the memory is O(n).

5. If engagement scores update frequently (every few seconds), what data structure would be more efficient than a sorted linked list?

A binary heap (max-heap): insertion, removal of the maximum and update of a score are O(log n), and the maximum is read in O(1). To update a score we also need a dictionary post_id -> position in the heap. If we also need the top k posts in order, a balanced binary search tree is another good choice.

More complexity analysis and reflection

Enqueue, RemoveById and UpdateScore are O(n), because they can walk the whole list, while DequeueMax and PeekMax are O(1). GetTopK is O(k). RefreshAll and DecayOlderThan are O(n^2), because they insert again all the n posts and every insertion is O(n). The memory is O(n).

UpdateScore and RefreshAll compute the score again from likes, comments and shares, so they remove the decay of the old posts (test "refresh after decay"). To keep the decay we would need to store it in the post.

Test set

The posts of the tests have the scores of the lab example: Post3 95 (the only old post), Post1 82, Post2 47 and Post4 23. For the bonus the random numbers start from a fixed seed, so the test always gives the same result. All the tests print OK.

Case	Operations	Expected	Got
lab example	initial queue	3:95, 1:82, 2:47, 4:23	3:95, 1:82, 2:47, 4:23
lab example	UpdateScore(Post2, +10 likes)	3:95, 1:82, 2:57, 4:23	3:95, 1:82, 2:57, 4:23
lab example	DecayOlderThan(50, 0.8)	1:82, 3:76, 2:57, 4:23	1:82, 3:76, 2:57, 4:23
core	PeekMax, then DequeueMax	Post1, Post1	Post1, Post1
core	GetTopK(2) after DequeueMax	Post3, Post2	Post3, Post2
k bigger than the queue	GetTopK(10) with 3 posts	3 posts	3 posts
k = 0	GetTopK(0)	empty array	empty array
empty queue	DequeueMax, PeekMax	NULL, NULL	NULL, NULL
post not in the queue	UpdateScore(9, +5 likes)	False	False
post goes to the top	UpdateScore(Post4, +50 comments)	Post4 first, score 123	Post4 first, score 123
refresh after decay	DecayOlderThan(50, 0.8), then RefreshAll	Post3 back to 95	Post3 back to 95
no old post	DecayOlderThan(0, 0.8)	nothing changes	nothing changes
equal scores	3 posts with score 10	arrival order 1, 2, 3	1, 2, 3
bonus	DualEnqueue of scores 101 and 100	101 viral, 100 normal	101 viral, 100 normal

Case	Operations	Expected	Got
bonus	DualDequeue, viral queue empty	from the normal queue	from the normal queue
bonus	DualDequeue, both queues empty	NULL	NULL
bonus	1000 DualDequeue, 1000 viral and 1000 normal posts	about 700 viral	675 viral

FINAL QUESTION

Based on your previous work, propose how you might combine in your future semester project doubly linked lists, stacks/queues and priority queues. How would these structures work together to build a complete social media feature (e.g., "Trending Feed with Friend Activity")?

In the semester project we have to build the backend of a social network, where the users and their relations (friends, followers) are saved in a graph. We have not chosen which social network yet, but the project asks for a feed ranking, recommendations and the analysis of viral posts, so the structures of this lab could work together like this:

- the priority queue ranks the posts by engagement score: with GetTopK we take the trending posts, and with DecayOlderThan the old posts go down, so we can also see which posts are becoming viral;
- the friends of the user are the neighbours of the user in the graph: their new posts and the trending posts become the feed, a doubly linked list where the user moves forward and backward and where we count the views;
- the actions of the user (like, comment, share, follow) go on the stack, so they can be undone, and the notifications of the friends arrive in the queue and are processed in order;
- every like, comment or share calls UpdateScore, so a post can move up in the priority queue, and a new follow adds an edge to the graph.

We have to write the graph algorithms ourselves, and we can use these structures there too: for example BFS uses a queue to find the friends of friends.